

4.1 Functional Requirement Models

UC-1: User Registration

The customer creates a new account in the online pharmacy system by entering personal information such as full name, email address, phone number, and password. Then The system creates a new account. After successful registration, the customer can access the system services such as ordering medicines, uploading prescriptions, and tracking orders.

UC-2: User Log in

The user accesses the login page and enters their email/password in the login form. The system processes the login request by validating the input and checking the credentials against the stored data in the database. If the credentials are correct, The user is then granted access to their account dashboard based on their role (customer, pharmacist, or admin). Otherwise, the system rejects the login attempt and shown generic error message without revealing sensitive details.

UC-3: Search Products and View Details

The customer searches for a medicine or a product by entering its name or a keyword in the search bar, then the system processes the search request and displays a list of matching results. The customer selects a product from the search results to view its details like the price, product description, dosage instructions, availability in inventory and prescription requirements when required. Finally, the customer may then add the product to the shopping cart or continue browsing.

UC-4: Upload Prescription

A registered customer uploads a digital copy of a medical prescription. The system validates the file format and ensures prescription validity. The system then authorizes the user, stores the prescription securely, and notifies a pharmacist for review. The pharmacist approves or denies the prescription.

UC-5: Checkout & Payment

The customer completes the medicine purchase process by reviewing the shopping cart, selecting a delivery address and payment method, and entering payment information. The system processes the payment request through the payment gateway, generates the order confirmation, updates the inventory, and stores the order information. Finally, the customer receives the confirmation message and payment receipt.

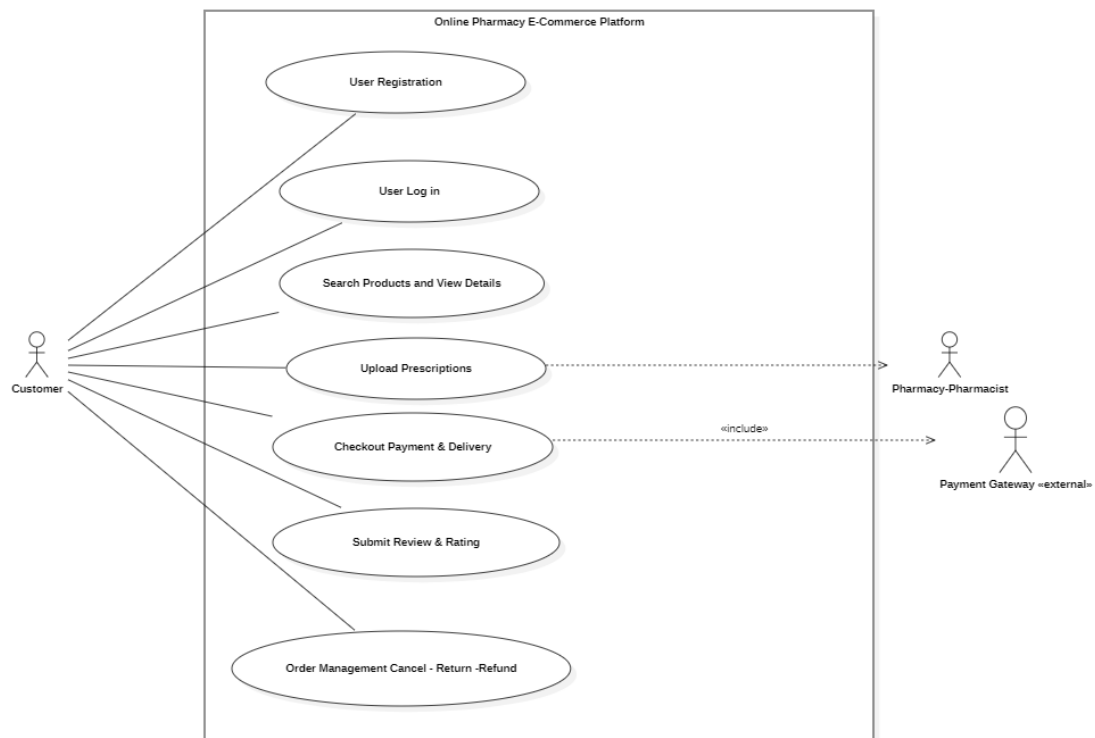
UC-6: Submit Review & Rating

A registered user selects a specific medicine from the system, clicks on "Write Review", chooses a rating between 1 and 5 stars, writes a text review, and submits it. The system then saves the review and displays it along with other reviews for that medicine

UC-7: Request Refund / Cancel / Return Order

an authenticated User submits a request to cancel, return, or request a refund, the system receives the Order ID and Refund Details, verifies Order Ownership, and checks business Eligibility. Once verified and Canonicalized, the request is routed to a Pending-Approval state for Admin review, finalizing with a Log Committed upon approval, while any ownership or validation failure immediately rejects the request, rolls back changes, and displays a generic error.

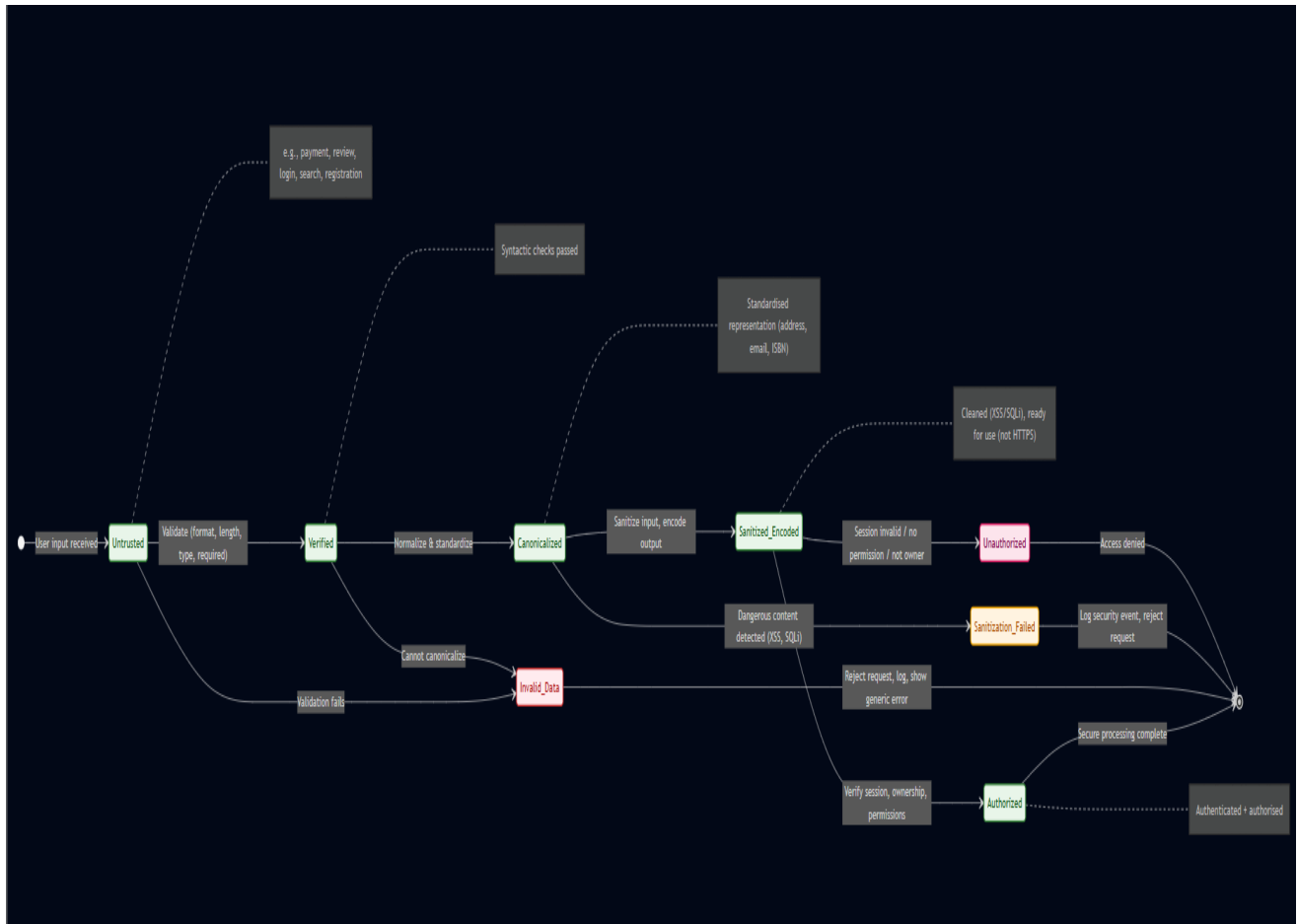
Use case diagram



4.2 Input Trust State Machine

State	Definition	Entry Condition	Exit Condition	Transition To
Untrusted	Raw input received from an external source such as a user, API, or file upload. The data is not yet trusted.	Data arrives from an external entity	Validation checks are initiated	Verified, Rejected
Verified	Input has passed syntactic validation checks including type, format, length, range, and required fields.	Syntactic validation succeeds	Normalization or encoding process is applied	Canonicalized, Sanitized/Encoded, Rejected
Canonicalized	Input is transformed into a standard and unambiguous internal representation.	Normalization succeeds	Authorization validation is performed	Authorized, Rejected
Sanitized / Encoded	Input has been cleaned, encoded, parameterized, or encrypted for secure usage.	Sanitization or encoding succeeds	Authorization validation is performed	Authorized, Rejected
Authorized	Input is confirmed to belong to the authenticated session/user and complies with business rules and access control policies.	Authorization succeeds	Processing or storage is completed	End (Final State)
Rejected	Input is unsafe, invalid, or fails security validation and cannot proceed further.	Any validation or authorization check fails	Error handling and audit logging occur	End (Final State)

state machine diagram



4.3

UC-1: User Registration

Field	Content
UC-ID	UC-1
Name	User Registration
Actors	Primary Actor: Customer Secondary Actor: Authentication System / User Database
Brief description	The customer creates a secure account in the online pharmacy system to access pharmacy services and manage orders safely.
Security preconditions	HTTPS connection is enabled. Registration requests are accepted only through secure channels. CAPTCHA protection is enabled to reduce automated registrations. Password policy requirements are enforced.
Trust state required on entry	• Full Name - Untrusted; will be verified internally • Email Address - Untrusted; will be verified internally • Phone Number - Untrusted; will be verified internally • Password - Untrusted; will be verified internally
Main success scenario (secure flow)	1. The customer opens the registration page and enters personal information including full name, email address, phone number, and password. The input is initially treated as untrusted. 2. The system validates the registration input by checking the email format, phone number format, password complexity requirements, and allowed characters. The input becomes Verified. (Principle: Complete Mediation) 3. The system canonicalizes the validated input by trimming spaces and normalizing text formatting. The input becomes Canonicalized. (Principle: Defense in Depth) 4. The system sanitizes all text inputs to prevent SQL injection and malicious scripts before processing the request. The input becomes Sanitized. (Principle: Defense in Depth) 5. The system checks whether the email address already exists in the database. 6. The password is hashed securely before storage in the database. (Principle: Least Privilege) 7. The system creates the customer account and logs the registration event securely. 8. The customer receives a confirmation message indicating successful account creation.
Alternative / error flows	If the email address format is invalid, the system rejects the registration request and displays a generic error message. If the password does not meet the required security policy, the system requests a stronger password. If the email address already exists, the system denies account creation and informs the customer without exposing sensitive information. If malicious input or SQL injection patterns are detected, the system blocks the request and logs the suspicious activity internally.
Security postconditions	• Customer account is securely stored in the database. • Password is stored as a hashed value • Registration activity is logged for auditing purposes. • Sensitive system information is not exposed to the customer.
Associated abuse cases	1. SQL Injection in registration fields An attacker attempts to inject malicious SQL commands through registration inputs to manipulate the database. The system prevents this using input validation, sanitization, and parameterized queries. 2. Weak password attack An attacker attempts to exploit weak passwords to gain unauthorized access. The system prevents this by enforcing strong password complexity requirements. 3. Automated fake account creation An attacker attempts to create multiple fake accounts automatically. The system prevents this using CAPTCHA and validation mechanisms.
Principles employed	• Complete Mediation. • Defense in Depth. • Least Privilege. • Fail-Safe Defaults.

UC-2: User Login

Field	Content
UC-ID	UC-2
Name	User Login
Actors	customer, Database, Server,Audit log
Brief description	The login use case allows registered customers to access their accounts by entering the correct username/password, followed by entering an MFA code to verify the user's identity.
Security preconditions	User Already has a registered account, user credentials are stored in database using hashing algorithms instead of Plaintext passwords, secure session management is enabled ,the system sanitizes login input to prevent injection attack.
Trust state required on entry	• Username/Email — Untrusted; verified internally by the system. • Password — Untrusted; verified internally against the hashed password stored in the database. • MFA Code — Untrusted; verified internally by the authentication service. • Session Token — Verified.
Main success scenario (secure flow)	1-User enters Username/password through the Login page 2- System validates andsanitizes user input.principle:complete Mediation 3 -System establishes a secure authenticated connection with the server. 4-System check whether the account exists and active. 5-System compares the password with the hashed password stored in the database. 6-System detects suspicious or repeated failed login attempts. 7-MFA verification is requested . 8- System creates a secure session for authenticated user. 9-User is granted access only to authorized resources based on role privileges. 10-System logs the login activity for auditing, and monitoring purposes.
Alternative / error flows	1-Invalid Input If the user entered input contains invalid characters, or malicious code, or incorrect formatting the system will rejects the request, sanitizes the input, and displays a generic error message. 2-Incorrect Credentials If the username or password is incorrect, the system denies access and displays a generic error message without specifying which field is incorrect. 3-Multiple Failed Login Attempts If the system detected multiple failure login attempts , the system temporarily locks the account, or applies rate limiting to prevent brute force attacks. 4-MFA Verification Failure If the MFA code is invalid or expired, the system rejects the authentication request, and logs the failed attempt. 5-Database or Server Failure If the authentication service ,or database becomes unavailable, the system denies login requests safely ,and records the error internally without exposing technical details to the user.

Security postconditions	• restricted User access based on assigned role privileges. • all Login activity for user is recorded in the audit log. • Failed authentication attempts are logged and monitored. • Session cookies are generated securely. • Sensitive credentials are never stored or exposed in plain text. .the system show generic error messages, no sensitive system or databases details are exposed to the user • Suspicious login activity may trigger temporary account lockout or additional verification.
Associated abuse cases	Cross-Site Scripting (XSS) An attacker injects malicious javascript code into login input fields to steal .session data or manipulate the application interface The secure flow mitigates is input sanitization, and format validation. 2. Brute Force Attack An attacker repeatedly attempts different password combinations to gain unauthorized access to the system. The secure flow mitigate is using rate limiting, temporary account lockout, and failed attempt monitoring. 3. SQL Injection An attacker inserts malicious SQL statements into login input fields to gain unauthorized access to the database. The secure flow mitigates is input validation, and sanitization.
Principles employed	• Least Privilege • Faile Safe Default • Economy of Mechanism • Defense in Depth • Complete Mediation

UC-3 : Search Products and View Details

Field	Content
UC-ID	UC-3
Name	Search Products and View Details
Actors	Primary Actor: Customer Secondary Actor: Inventory System / Product Database
Brief description	The customer searches for a product and view its details, for easier and more convenient shopping.
Security preconditions	HTTPS connection, publicly available search service with rate limiting, product data is retrieved only through controlled backend API, no direct database access is allowed from the client side.
Trust state required on entry	- Search Keyword - Untrusted; will be verified internally - Selected Product ID - Untrusted; will be verified internally - Filter values - Untrusted; will be verified internally - Pagination values - Untrusted; will be verified internally
Main success scenario (secure flow)	- The customer enters a product name or keyword in the search bar. The customer input is initially treated as untrusted. - The system validates the search input by checking length, allowed characters, and rejecting malicious patterns. The input becomes verified. (Principle: Complete Mediation) - The system canonicalizes the validated input by trimming spaces and normalizing the text format. The input becomes canonicalized. (Principle: Defense in Depth) - The system sends the search request to the database using parameterized queries to prevent SQL injection. The input becomes Sanitized for database processing. (Principle: Defense in Depth) - The database returns matching products to the system. - The system encodes product names and descriptions before displaying them to the customer to prevent XSS attacks. The output becomes sanitized/encoded for display. (Principle: Complete Mediation) - The customer selects a product from the search results. - The system validates the selected product ID and confirms that the product exists and is available for viewing. The product data becomes Authorized. (Principle: Fail-Safe Defaults) - The system displays the product details, like description, dosage information, price, availability, and prescription requirements.
Alternative / error flows	If excessive search requests are detected from the same user or IP address, the system applies rate limiting. If the search input contains invalid characters or exceeds the allowed length, the system rejects the search request and displays a generic message. If SQL injection patterns are detected, the system blocks the search request, logs the event and prevent database execution. If the selected product ID is invalid, the system displays (Product not found) message without exposing internal system details. If a database error happens during the search process, the system displays a generic error message and logs the technical details internally.
Security postconditions	- Search input was validated and canonicalized. - Database queries were executed using parameterized queries. - Output was displayed encoded. - Suspicious attempts were logged. - No internal database or system errors were exposed to the customer.
Associated abuse cases	Brute force attack An attacker attempts to send a large number of search requests in a short amount of time to overwhelm the system causing Denial of Service (DOS) attack. The system prevents this by using rate limiting.

	2. SQL injection in the search field An attacker attempts to enter malicious SQL code in the search bar to manipulate database queries or access unauthorized data. The system prevents this by using input validation and parameterized queries.
Principles employed	• Complete Mediation. • Defense in Depth. • Fail-Safe Defaults. • Economy of Mechanism.

UC-4: Upload Prescription

Field	Content
UC-ID	UC-4
Name	Upload Prescription
Actors	Primary Actor: Customer Secondary Actor: Pharmacist, System
Brief description	Allow an authenticated customer to upload a medical prescription file. The system scans viruses, verifies the details, stores it securely, and sends it to the pharmacist for approval.
Security preconditions	1. User must be logged in (Authenticated). 2. Secure HTTPS connection is established. 3. Antivirus scanning tools are active on the server.
Trust state required on entry	• Prescription File: Untrusted (Potential malware). • User Session: Verified (Trusted). • File Content: Untrusted until scanned.
Main success scenario (secure flow)	1. Customer clicks "Upload Prescription". 2. System displays upload page with format and size limits. 3. Customer selects a file from the device. 4. System receives the file (State: Untrusted). 5. System validates file type, size, and scans for malware (State: Validated Input / Scanned Clean). 6. System verifies prescription details using OCR, readability check, and expiry validation (State: Content Verified). 7. System checks user authentication and authorization (State: Authenticated User / Authorized Request). 8. System stores the file securely (Encrypted) and links it to the user (State: Stored Securely). 9. System logs the upload audit event (Event ID, User, Timestamp) (State: Audited - Upload). 10. System notifies the pharmacist for review. 11. Pharmacist reviews the prescription. 12. If approved, pharmacist confirms approval (State: Authorized). 13. System logs pharmacist approval event (State: Audited - Approval). 14. System attaches the prescription to the order. 15. System updates status as approved and notify the user to proceed to checkout.
Alternative / error flows	• Invalid File: If the file type is wrong or infected, the system rejects it and shows a generic error message.

	• Invalid Prescription: If the prescription is expired or unclear, the system asks the user to reupload. • Unauthorized Access: If the user lacks permission, access is denied and a warning is logged. • Pharmacist Rejection: If rejected, the system notifies the user to upload a corrected version. • Database Error: If storage fails, the system shows a generic error and logs the technical details internally.
Security postconditions	• File is stored in an encrypted format. • Audit trail is created successfully for both upload and approval events. • Access is restricted to the authorized user and pharmacist only.
Associated abuse cases	1. Malware Upload: An attacker tries to upload a malicious file. Prevented by malware scanning. 2. Unauthorized Access: An attacker tries to upload a prescription for another user. Prevented by session and ownership checks.
Principles employed	• Fail-Safe Defaults: Rejects unsafe files and denies access by default. • Defense in Depth: Uses validation, scanning, and encryption layers. • Least Privilege: Access restricted to owner and pharmacist only. • Complete Mediation: Verifies user authorization at every step. • Separation of Duty: User uploads and pharmacist approves.
Designer's Rationale	Input validation, malware scanning, authentication, and authorization are performed sequentially to ensure the file is safe before processing and accessible only by authorized users. This prevents malicious uploads from compromising the server and protects pharmacist review workflows securely.

UC-5: Checkout & Payment

Field	Content
UC-ID	UC-5
Name	Checkout & Payment
Actors	Primary Actor: Customer Secondary Actors: Pharmacy System, Payment Gateway
Brief description	This use case allows authenticated customers to securely complete medicine purchases while ensuring the confidentiality and integrity of payments and protecting against unauthorized transactions.
Security preconditions	• User must be authenticated. • Secure HTTPS/TLS connection must be active. • User session must be valid. • Payment gateway must be trusted and verified. • CSRF token must be valid. • The customer account must not be suspended or restricted.
Trust state required on entry	• Shopping Cart Data – Authorized; verified to belong to the authenticated user session. • Session ID – Authorized; validated as an active authenticated session. • Delivery Address – Untrusted; will be validated and canonicalized internally. • Payment Information – Untrusted; will be validated and encrypted internally. • CSRF Token – Verified; checked for validity before processing the request.
Main success scenario (secure flow)	• Step 1: System receives "shopping cart data" (Trust State: Untrusted). System validates cart belongs to authenticated session and contains valid items → cart data becomes Authorized. (Principle: Complete Mediation) • Step 2: System receives "delivery address" and "payment method selection" (Trust State: Untrusted). System validates address format and required fields, verifies payment method is available → address and method become Verified. (Principle: Fail-Safe Defaults) • Step 3: System receives "validated address" (Trust State: Verified). System canonicalizes address format, normalizes field order, standardizes abbreviations → address becomes Canonicalized. (Principle: Defense in Depth) • Step 4: System receives "payment information" (card number, expiry, CVV) (Trust State: Untrusted). System validates card format, expiry date not past → payment info becomes Verified. (Principle: Complete Mediation) • Step 5: System receives "verified payment information" (Trust State: Verified). System encrypts payment data using HTTPS/TLS, sends to Payment Gateway → payment request becomes Sanitized/Encoded. (Principle: Defense in Depth) • Step 6: Payment Gateway receives "encrypted payment request" (Trust State: Encrypted). Gateway authorizes the transaction with the bank, checks funds, and fraud rules → transaction becomes Authorized. (Principle: Least Privilege) • Step 7: System receives "payment approval" (Trust State: Authorized). System generates order confirmation, requests Pharmacy System to update inventory and store data → output becomes Sanitized/Encoded before display. (Principle: Separation of Duty) • Step 8: System sends "receipt notification" (Trust State: Sanitized). User receives confirmation message and payment receipt. (Principle: Psychological Acceptability)

Alternative / error flows	• Invalid Shipping Address System rejects checkout request. Generic error shown. Details logged internally. User returns to Step 2. (Principle: Fail-Safe Defaults) • Invalid Payment Information System blocks transaction. Generic error shown. Failed validation logged. User returns to Step 4. (Principle: Complete Mediation, Fail-Safe Defaults) • Payment Authorization Failure System receives rejection from Gateway Order not created. Failure logged. User returns to Step 4 to try again or use different payment methods. (Principle: Defense in Depth) • Session Expiration System terminates checkout process. Redirects to login page. User must re-authenticate. (Principle: Fail-Safe Defaults) • Canonicalization Failure The system cannot standardize the address format. Request rejected. → Generic error shown. Technical details logged internally. User returns to Step 2. (Principle: Defense in Depth)
Security postconditions	• Audit log entry created with transaction ID. • No plain-text payment data stored in the system. • Output encoded securely before display. • Failed payment attempts are securely logged for monitoring and auditing purposes. • Order confirmation generated and stored • No internal system details exposed to the user. • Inventory decreased atomically.
Associated abuse cases	• The attacker attempts to tamper with payment parameters to modify transaction values. → Server-side validation and payment authorization checks prevent unauthorized modification. • Attacker attempts session hijacking during checkout. → Secure session validation and HTTPS/TLS encryption protect the authenticated session. • Attackers attempt to use stolen payment information. → Payment authorization and verification mechanisms detect unauthorized transactions. • Attacker attempts unauthorized order manipulation. → Access control and authenticated session checks restrict unauthorized actions.
Principles employed	• Least Privilege • Defense in Depth • Complete Mediation • Fail-Safe Defaults • Separation of Duty • Psychological Acceptability

UC-6 : Submit Review & Rating

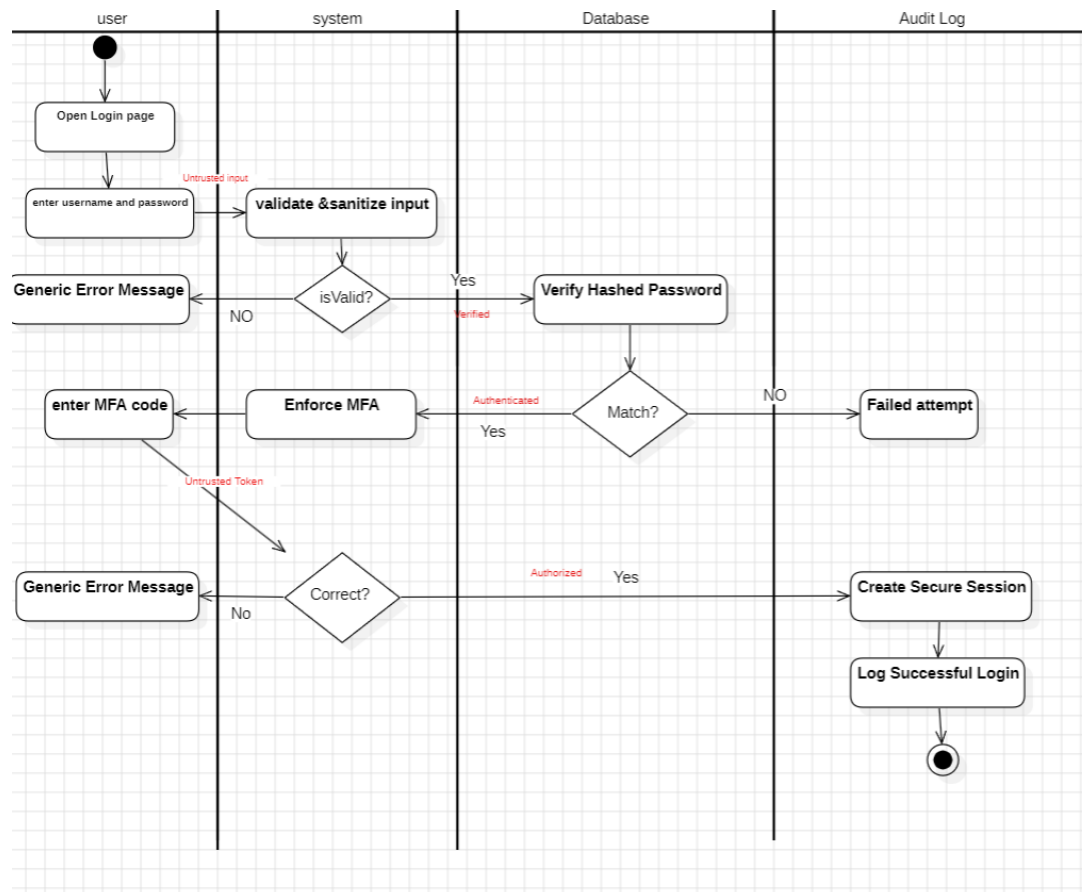
Field	Content
UC-ID	UC-6
Name	Submit Review & Rating
Actors	Registered User (primary), Pharmacy System (secondary for verification)
Brief description	User submits a rating (1-5 stars) and a text review for a medicine they have purchased.
Security preconditions	1. User is authenticated (active session) 2. User has a completed order containing this medicine (verified via Order Service) 3. User has not already reviewed this medicine for the same order
Trust state required on entry	- Rating (int) → Untrusted - Review text (String) → Untrusted
Main success scenario (secure flow)	1. System receives rating and review text → both Untrusted 2. Validate rating is between 1-5 → becomes Verified (Principle: Complete Mediation) 3. Check length of review text (e.g., max 500 chars) → becomes Verified (Principle: Defense in Depth) 4. Sanitize review text: remove <script>, HTML tags, JavaScript events → becomes Sanitized (Principle: Defense in Depth) 5. Filter profanity: if detected → reject or mask → becomes Authorized (Principle: Fail-Safe Defaults) 6. Verify user actually purchased this medicine (via Order Service) → becomes Authorized 7. Save review to database using parameterized queries (to prevent SQL injection) 8. When displaying review, apply output encoding (e.g., < → &lt;)
Alternative / error flows	- Rating out of range (1-5) → reject with error - Review text contains <script> or XSS patterns → reject + log security event - Profanity detected → reject with "inappropriate content" OR mask with *** - User hasn't purchased medicine → reject with "You can only review purchased medicines" - Duplicate review → reject with "You already reviewed this medicine"
Security postconditions	- Review stored safely in database (no executable code) - Security log entry created (user ID, medicine ID, timestamp) - Review displayed with proper encoding to prevent XSS
Associated abuse cases	1. XSS Attack: User submits <script>alert('XSS')</script> → sanitized before storage 2. Fake reviews: User who never purchased submits review → blocked by precondition 3. Spam: User submits hundreds of reviews → limited by "one review per order" 4. Profanity/Harassment: User submits offensive text → filtered or rejected
Principles employed	- Complete Mediation: Every input is validated before processing - Defense in Depth: Multiple checks (validation, sanitization, output encoding) - Fail-Safe Defaults: Suspicious input is rejected, not accepted - Least Privilege: Review service only has write access to reviews table - Psychological Acceptability: User receives clear error messages

UC-7 : Request Refund / Cancel / Return Order

Field	Content
UC-ID	UC-7
Name	Request Refund / Cancel / Return Order
Actors	User Primary, Admin Secondary
Brief description	Allows users to securely cancel orders, return items, or request refunds while ensuring strict authorization checks, transaction integrity, and full audit logging.
Security preconditions	1. User must be fully authenticated with an active, valid session token. 2. Request must contain a valid, non-expired Anti-CSRF token. 3. Connection must be strictly encrypted via HTTPS/TLS.
Trust state required on entry	Order ID: Untrusted; context and ownership must be validated. Refund Amount or Reason: Untrusted; must be verified against business rules and sanitized.
Main success scenario (secure flow)	1. System receives action request containing Order ID and contextual details. All parameters enter Untrusted state. 2. Validate Order Ownership and Integrity. System checks if the logged-in User actually owns the specified Order ID and ensures the order status permits this action. Inputs become Verified. Principle: Complete Mediation 3. Check Authorization and Eligibility. System validates user privileges and verifies if the item is within the allowable window. Request becomes Authorized. (Principle: Least Privilege) 4. Sanitize Contextual Input. Canonicalize and clean text entry for the return or refund reason to prevent injection vectors. Text fields become Canonicalized. (Principle: Defense in Depth) 5. Securely Record Request and Trigger Admin Approval. System flags the order, stores transaction states securely, and routes high-value refund actions to the Admin actor for explicit approval. Transaction status becomes Pending-Approval. Principle: Separation of Duty 6. Generate Immutable Audit Log. System automatically writes a detailed, tamper-resistant log entry containing Event ID, Actor ID, Timestamp, and Action Type. Log entry is securely Committed. Principle: Defense in Depth Flow A: Order Ownership or Auth Check Fails. User tries to access or refund someone else's order ID. System triggers the Block Unauthorized Access flow, terminates the session immediately, logs a high-severity security alert, and displays a generic error message. Principle: Fail-Safe Defaults Flow B: Ineligible for Refund or Cancel. Order already shipped or past expiry date. System rejects the request, rolls back any state changes, and displays a business-rule violation message without modifying payment states. Principle: Fail-Safe Defaults
Alternative / error flows	Flow A: Order Ownership or Auth Check Fails. User tries to access or refund someone else's order ID. System triggers the Block Unauthorized Access flow, terminates the session immediately, logs a high-severity security alert, and displays a generic error message. Principle: Fail-Safe Defaults Flow B: Ineligible for Refund or Cancel. Order already shipped or past expiry date. System rejects the request, rolls back any state changes, and displays a business-rule violation message without modifying payment states. Principle: Fail-Safe Defaults
Security preconditions	1. An immutable cryptographic audit log is generated and stored for compliance. 2. Financial or Order states are updated atomically to prevent race conditions or double-refund exploits. 3. Admin is securely notified of the pending refund request with zero exposure of underlying internal database structures.
Associated abuse cases	1. Insecure Direct Object Reference IDOR or Parameter Tampering: An attacker alters the Order ID in the request payload to cancel or refund another user's purchase. Neutralized by the mandatory Validate Order Ownership step. 2. Double Refund Exploit or Race Condition: An attacker sends multiple concurrent refund requests rapidly to trick the system into paying out twice. Neutralized by atomic database operations and strict state locking during the secure flow.
Principles employed	Complete Mediation, Separation of Duty, Fail-Safe Defaults, Least Privilege, Defense in Depth.

4.4 Activity Diagram

- UC-2:User Login



- UC4: Upload Prescription.

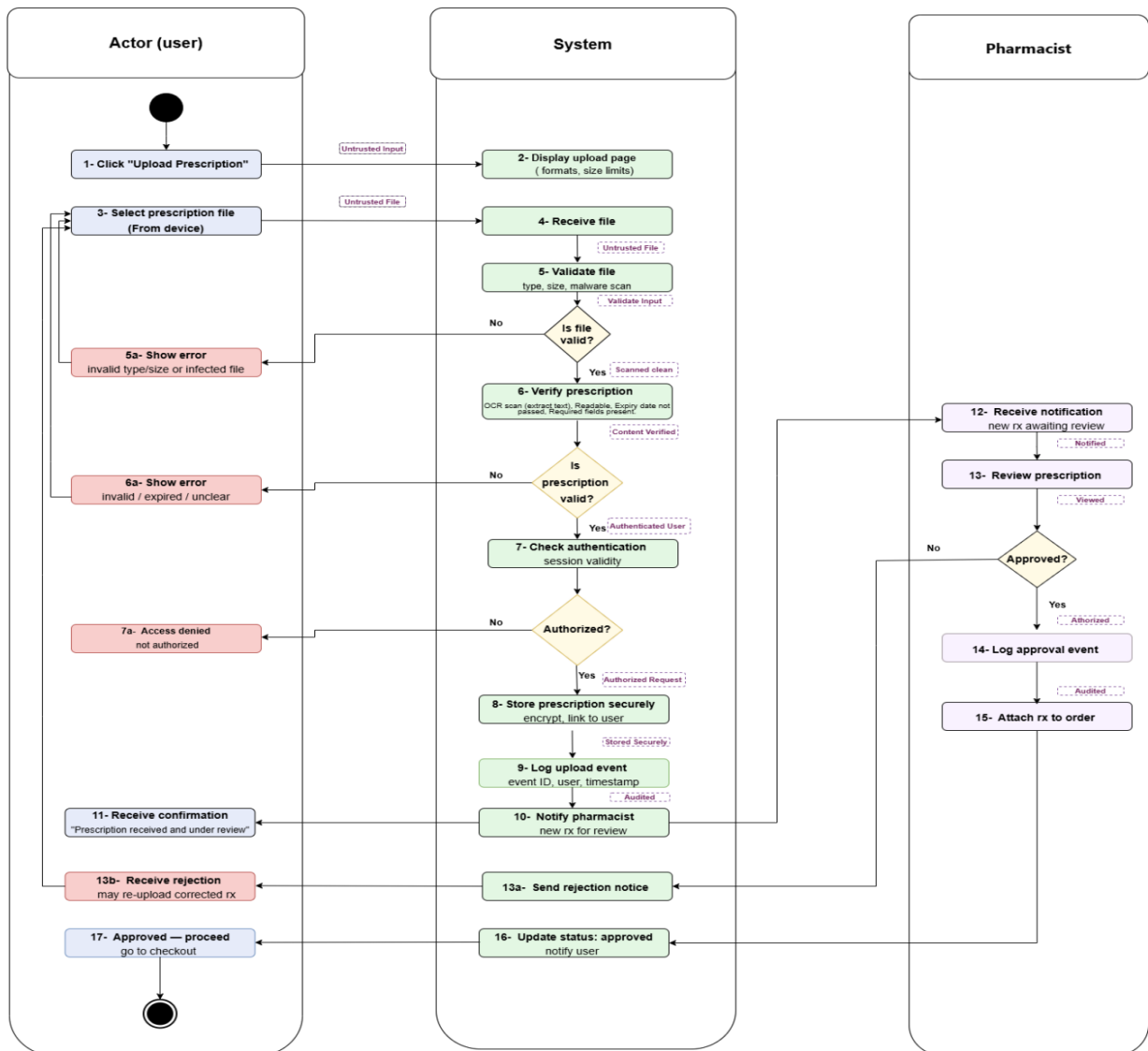
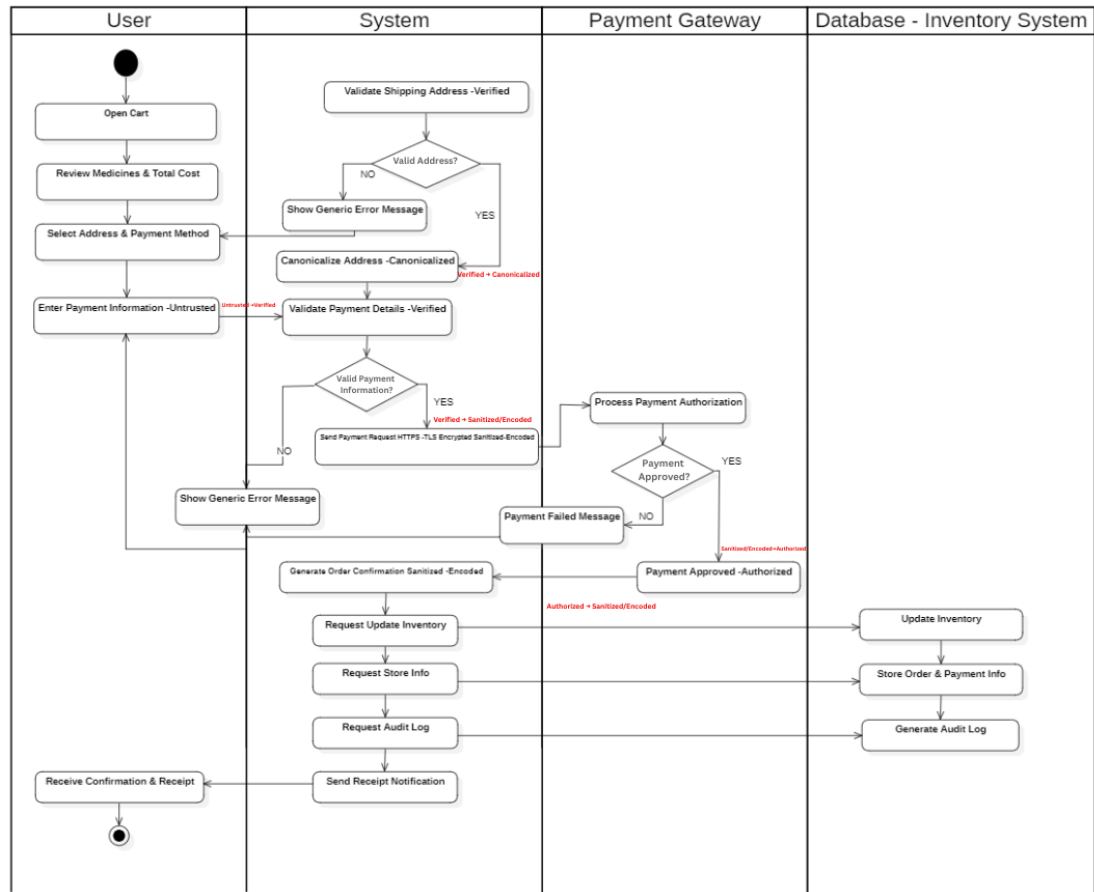


Figure : Secure Upload Prescription Activity Diagram

UC-5: Checkout & Payment



4.5

The biggest challenge in designing the online pharmacy platform was balancing usability with security. Since the platform is intended for regular customers, the system needed to remain simple, fast, and convenient to use. Features such as product search, login, prescription upload, and checkout had to feel smooth and user-friendly without overwhelming the user with unnecessary security steps. At the same time, these features process sensitive information and could become targets for attacks such as SQL injection, XSS, brute-force attempts, malicious file uploads, or unauthorized access to prescription and payment data.

One example of this trade-off appeared in the search functionality. Users expect the search feature to accept flexible keywords and provide quick results, but unrestricted input could expose the system to SQL injection or abuse through automated requests. To reduce this risk, the design included input validation, parameterized queries, output encoding, and rate limiting while still keeping the search process simple from the customer's perspective. Similar trade-offs also appeared in the login and checkout processes, where stronger security controls such as MFA, session validation, and secure payment handling improve protection but may slightly reduce convenience for some users.

If the system had to fully comply with standards such as PCI-DSS or HIPAA, additional protections would need to be implemented. Payment information would require stricter encryption, stronger access control, secure audit logging, and continuous monitoring of financial transactions. Prescription and healthcare-related data would also require tighter privacy controls, limited employee access, and secure storage mechanisms to ensure confidentiality. In a real production environment, the system would also require regular security testing, vulnerability assessments, and compliance reviews to maintain trust and protect sensitive customer information.

To achieve this level of security in a structured way, we designed an Input Trust State Machine (Section 4.2). This model defines how all user inputs – from registration and search to payment and reviews – evolve from untrusted to fully authorized data. The machine provided the following key **benefits to our design**:

Early identification of security gaps :Revealed missing checks, e.g., UC-6 needed Authorized state (purchase verification) ,UC-3 needed Canonicalized step.

Uniform security across use cases: All inputs follow the same sequence:

Untrusted → Verified → Canonicalized → Sanitized_Encoded → Authorized.

Support for defense in depth :Multiple independent layers (validation, canonicalization, sanitization, authorization) block attacks even if one layer is bypassed.

Clear classification of failures :Distinguishes *Invalid_Data*, *Sanitization_Failed*, and *Unauthorized* for faster incident response.

Improved code reviews :Acts as a security checklist; missing steps become obvious improvement candidates.

Enhanced documentation :One high-level diagram replaces per-use-case explanations, aiding both instructor evaluation and team understanding.